

RAPPORT TECHNIQUE COMPLET

Enterprise Data Warehouse

Architecture Medallion — Bronze · Silver · Gold

□ **BRONZE**
Ingestion Brute

□ **SILVER**
Nettoyage & Qualite

□ **GOLD**
Modele en Etoile

PostgreSQL · Python · SQLAlchemy · psycopg2 · Pandas

Consommation : Power BI · SQL Ad-Hoc · Machine Learning

Juin 2025

1. SYNTHÈSE EXECUTIVE

Ce projet constitue une implementation complete, de bout en bout, d'un Entrepot de Donnees (Data Warehouse) d'entreprise. Il repose sur la methodology Medallion Architecture, qui organise les donnees en trois couches progressives de raffinement : Bronze (brute), Silver (nettoyee) et Gold (decisionnelle). L'ensemble est construit sur PostgreSQL comme moteur de base de donnees relationnelle, orchestre par Python, et produit un modele en etoile (Star Schema) pret pour la consommation analytique dans Power BI.

L'objectif metier central est de transformer des fichiers CSV transactionnels issus de systemes CRM et ERP en un referentiel analytique fiable, performant et totalement automatise, capable de repondre aux besoins de reporting commercial, d'analyse ad-hoc et potentiellement de Machine Learning.

Chiffres cles du projet

Metrique	Detail	Valeur
Sources CSV	Fichiers transactionnels et referentiels	6 fichiers
Tables Bronze	Tables d'atterrissage en VARCHAR brut	6 tables
Tables Silver	Tables nettoyees et typees	6 tables
Vues Gold	Modele en etoile (2 dimensions + 1 fait)	3 vues
Scripts Python	Extract.py, Transform.py, Load.py, Orchestrateur.py	4 scripts
Scripts SQL	6 Silver + 3 Gold + 1 Audit	10 scripts

2. ARCHITECTURE GLOBALE

L'architecture suit strictement le paradigme Medallion Architecture (aussi appelee Lambda Light Architecture dans certaines implementations). Ce modele est une reference industrielle pour la construction de Data Lakehouses et Data Warehouses modernes. Il separe les responsabilites en zones d'isolation etanches, garantissant la tracabilite, la reutilisabilite et la robustesse de chaque etape du pipeline de donnees.

2.1 Flux de donnees bout-en-bout



2.2 Principes d'ingenierie logicielle appliques

- Idempotence totale : chaque script peut etre relance autant de fois que necessaire sans corrompre la base (DROP TABLE/VIEW IF EXISTS + recreation complete).
- Ancrage dynamique des chemins : utilisation de pathlib.Path(__file__).resolve() pour eviter les chemins en dur et rendre le projet portable.
- Transaction atomique Silver : tout le pipeline Silver tourne dans une transaction unique psycopg2. En cas d'echec sur un script, un ROLLBACK annule toutes les modifications partielles.
- Principe Fail-Fast Gold : si une vue dimensionnelle echoue a la compilation, le pipeline s'arrete immediatement via RuntimeError pour ne pas exposer une table de faits corrompue.
- Zero propagation de NULL : utilisation systematique de COALESCE() pour remplacer les valeurs manquantes par 'n/a' dans toutes les colonnes de filtrage des vues Gold.
- Variables d'environnement : la chaine de connexion PostgreSQL est lue depuis la variable DATABASE_URL via os.getenv(), ce qui exclut les credentials du code source.

3. COUCHE BRONZE — Zone d'Atterrissage

La couche Bronze est le point d'entree unique et immuable du pipeline. Son role est de photographier fidelement les sources, sans aucune interpretation ni transformation metier. Elle constitue l'historique technique auditable du systeme.

3.1 Role et responsabilites

- Absorber les fichiers CSV sources en preservant 100% des donnees brutes.
- Forcer le typage de toutes les colonnes en VARCHAR/TEXT pour eviter tout rejet de ligne lors de l'ingestion (dates mal formatees, separateurs regionaux, caracteres speciaux).
- Nettoyer et standardiser les en-tetes de colonnes : suppression d'espaces, passage en minuscules, remplacement d'espaces par des tirets bas (_).
- Injecter dans PostgreSQL sous le schema 'Bronze' avec if_exists='replace' pour garantir l'idempotence totale.

3.2 Script d'extraction : Extract.py

Le script utilise la bibliotheque Pandas pour lire les CSV et SQLAlchemy comme ORM pour ecrire dans PostgreSQL. L'ancrage du chemin est gere dynamiquement : si le script est execute depuis la racine du projet, il retrouve automatiquement le sous-dossier Bronze/datasets.

Sequence d'execution de Extract.py

- Detection automatique du dossier racine via pathlib.
- Scan de tous les fichiers *.csv dans Bronze/datasets/.
- Pour chaque fichier : lecture avec dtype=str (tout en texte), nettoyage des noms de colonnes, injection dans Bronze.<nom_table> avec remplacement complet.
- Journalisation du nombre de lignes inserees par table.

3.3 Tables de la couche Bronze

Table	Fichier Source	Contenu metier
Bronze.cust_info	cust_info.csv	Profils clients principaux : identifiant, cle metier, nom, prenom, statut marital, genre, date de creation
Bronze.cust_az12	CUST_AZ12.csv	Source CRM secondaire : identifiant alternatif (prefixe NAS), date de naissance, genre secondaire
Bronze.loc_a101	LOC_A101.csv	Geographie clients : identifiant (avec tirets), pays de residence (multiples synonymes possibles)

Table	Fichier Source	Contenu metier
Bronze.prn_info	prn_info.csv	Catalogue produits : identifiant, cle produit (prefixee), nom, cout, gamme, date de debut de validite
Bronze.px_cat_g1v2	PX_CAT_G1V2.csv	Referentiel des categories : identifiant, categorie principale, sous-categorie, statut maintenance
Bronze.sales_details	sales_details.csv	Transactions de ventes : numero de commande, cle produit, identifiant client, dates (commande/expedition/echeance), montant, quantite, prix

4. COUCHE SILVER — Nettoyage & Data Quality

La couche Silver est le coeur de la Data Quality du projet. Elle orchestre la transformation majeure en purgeant les anomalies, appliquant un typage SQL strict et redressant les erreurs mathématiques ou logiques. C'est cette couche qui garantit que chaque indicateur final repose sur des données 100% saines et conformes.

4.1 Orchestrateur Silver : Transform.py

Transform.py exécute une suite ordonnée de 6 scripts SQL dans une transaction unique gérée par psycopg2. Un mécanisme de Rollback automatique est intégré : si un script SQL échoue, toutes les modifications de la session sont annulées instantanément.

Ordre d'exécution du pipeline Silver

- 1. transform_bronze_to_silver_cust_info.sql — Clients principaux
- 2. transform_bronze_to_silver_prd_info.sql — Catalogue produits
- 3. transform_bronze_to_silver_sales_details.sql — Transactions de ventes
- 4. transform_bronze_to_silver_cust_az12.sql — CRM secondaire
- 5. transform_bronze_to_silver_px_cat_g1v2.sql — Catégories (pass-through)
- 6. transform_bronze_to_silver_loc_a101.sql — Géographie

4.2 Detail des transformations par table

4.2.1 Silver.cust_info — Clients principaux

Ce script applique les transformations les plus complexes du pipeline :

- TRIM() systématique sur toutes les colonnes textuelles pour supprimer les espaces parasites (source fréquente d'erreurs de jointure invisibles).
- Mapping logique du statut marital : M devient 'Married', S devient 'Single', toute autre valeur ou NULL devient 'n/a'.
- Mapping logique du genre : M devient 'Male', F devient 'Female', autres cas sécurisés en 'n/a'.
- Dedoublonnage par fenêtrage SQL : ROW_NUMBER() OVER(PARTITION BY cst_id ORDER BY cst_create_date DESC) attribue un rang à chaque version d'un client. Seul le rang 1 (profil le plus récent) est conservé via WHERE rn = 1.
- Transtypage final : cst_create_date::DATE force la conversion du VARCHAR Bronze vers le type DATE natif SQL.
- Exclusion des lignes avec cst_id IS NULL pour garantir l'intégrité référentielle.

4.2.2 Silver.cust_az12 — CRM secondaire

- Nettoyage de l'identifiant : suppression du préfixe 'NAS' via CASE WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid FROM 4).
- Conversion sécurisée de la date de naissance : validation de la longueur (10 caractères attendus format YYYY-MM-DD) avant conversion. Si corrompu ou vide, la date pivot de sécurité '1900-01-01'::DATE est appliquée.

- Standardisation du genre avec gestion large : UPPER(TRIM(gen)) LIKE 'F%' => 'Female', LIKE 'M%' => 'Male' pour capturer toutes les variantes textuelles possibles.

4.2.3 Silver.loc_a101 — Géographie

- Nettoyage de l'identifiant : REPLACE(cid, '-', '') supprime les tirets pour assurer la cohérence des jointures avec cust_info.
- Normalisation des pays : regroupement des synonymes US/USA/UNITED STATES => 'United States', DE/GERMANY => 'Germany'.
- Traitement des espaces insecables : REPLACE(cntry, chr(160), '') élimine les caractères invisibles encodés qui génèrent des faux doublons.
- Sécurisation des NULL et chaînes vides par retour à 'n/a'.

4.2.4 Silver.prđ_info — Catalogue produits (SCD Type 2)

Ce script est le plus sophistiqué du pipeline Silver, implémentant une gestion de l'historique des versions de produits :

- Extraction de la clé de catégorie : REPLACE(SUBSTRING(prd_key, 1, 5), '-', '_') isole les 5 premiers caractères de la clé brute pour construire cat_id, clé de jointure avec le référentiel des catégories.
- Extraction de la référence produit : SUBSTRING(prd_key FROM 7) supprime les 6 premiers caractères du préfixe technique pour exposer la référence métier pure.
- Nettoyage du coût : chaîne de NULLIF + NUMERIC + vérification négative/nulle/NULL, tout redresse à 0 en cas d'anomalie.
- Normalisation de la gamme : R => 'Road', M => 'Mountain', T => 'Touring', S => 'Other Sales', autres cas => 'n/a'.
- SCD Type 2 via LEAD() : la fonction LEAD(prd_start_dt, 1, '9999-12-31') calcule dynamiquement la date de fin de validité de chaque version produit en récupérant la date de début de la version suivante moins 1 jour. Le produit actif reçoit la date infinie '9999-12-31'. Seules les versions actives (prd_end_dt = '9999-12-31') sont conservées dans Silver.

4.2.5 Silver.px_cat_g1v2 — Catégories (Pass-Through)

Ce script est un transfert direct : les données du référentiel des catégories n'ont pas besoin de transformation. La logique est un simple CREATE TABLE ... AS SELECT * FROM Bronze.px_cat_g1v2, avec DROP/CREATE pour assurer l'idempotence. La valeur ajoutée est la centralisation de cette table dans le schéma Silver au même niveau de disponibilité que les autres tables.

4.2.6 Silver.sales_details — Transactions financières

Ce script applique une triple validation et un redressement en cascade à travers trois étapes (2 CTEs + SELECT final) :

- Étape 1 (CTE step1_abs) : Standardisation des clés textuelles via NULLIF(TRIM()), mise en quarantaine des dates brutes, application de ABS() sur tous les montants numériques (correction des négatifs) et sécurisation des NULL/vides via COALESCE à 0.
- Étape 2 (CTE step2_clean_defaults) : Élimination des zéros : toute quantité ou prix égal à 0 est redressé à 1 pour bloquer les divisions par zéro.

- Etape 3 (SELECT final) : Parsing securise des dates YYYYMMDD (validation de longueur = 8 caracteres avant TO_DATE), application des regles d'alignement comptable : Regle A : si sls_sales = 0, recalcul dynamique = Quantite * Prix. Regle C : si sls_sales existait, le prix est recalcule = Ventes / Quantite pour garantir la coherence mathematique exacte au centime pres.

5. COUCHE GOLD — Modele en Etoile Decisionnel

La couche Gold est la vitrine analytique du Data Warehouse. Elle consomme les donnees nettoyees de Silver pour les exposer sous la forme d'un modele en etoile (Star Schema) conforme a la methodologie de Kimball. Cette couche est implementee en Vues SQL dynamiques (et non en tables physiques), ce qui garantit que Power BI consulte toujours les donnees les plus recentes sans overhead de stockage supplementaire.

5.1 Orchestrateur Gold : Load.py

Load.py utilise SQLAlchemy (engine.begin() pour les transactions automatiques) et execute les scripts Gold dans un ordre strict respectant les dependances : dim_customer et dim_product doivent exister avant fact_sales qui les reference. Le principe Fail-Fast est applique : tout echec declenche un RuntimeError qui interrompt immediatement le pipeline.

5.2 Dimension Clients : Gold.dim_customer

Cette vue unifiee reconcilie trois sources Silver en une dimension client complete :

Champ BI	Type SQL	Source Silver	Regle metier
customer_id	BIGINT	Genere	Surrogate Key via ROW_NUMBER() OVER()
customer_key	VARCHAR	cust_info.cst_id	Cle technique interne du systeme source
customer_number	VARCHAR	cust_info.cst_key	Numero de compte business (nettoye des espaces)
first_name / last_name	VARCHAR	cust_info	Prenom et nom du client
gender	VARCHAR	cust_info + cust_az12	Cross-Source Fallback : source principale > CRM secondaire > 'n/a'
marital_status	VARCHAR	cust_info	Statut marital normalise (Married/Single/n/a)
country	VARCHAR	loc_a101	COALESCE(c.cntry, 'n/a') apres LEFT JOIN sur cst_key
birth_date	DATE	cust_az12	Date de naissance, securisee a 1900-01-01 si corrompue
create_date	DATE	cust_info	Date de creation du profil client en source

Logique critique — Cross-Source Fallback Gender : la colonne gender applique une cascade de priorite en 3 regles. Regle 1 : si le genre est valide dans cust_info (different de 'n/a'), il est conserve.

Regle 2 : s'il est 'n/a' en source principale mais present dans cust_az12 (le CRM secondaire), la valeur secondaire est utilisee. Regle 3 : si absent des deux sources, securisation avec 'n/a'. Cette logique garantit le taux de completude maximal de l'axe d'analyse genre.

5.3 Dimension Produits : Gold.dim_product

Champ BI	Type SQL	Source Silver	Regle metier
product_key	BIGINT	Genere	Surrogate Key via ROW_NUMBER() OVER()
product_id	INTEGER	prd_info.prd_id	Identifiant technique brut du systeme source
product_number	VARCHAR	prd_info.prd_key	Reference SKU metier (apres suppression du prefixe technique)
product_name	VARCHAR	prd_info.prd_nm	Libelle commercial complet du produit
category / sub_category	VARCHAR	px_cat_g1v2	COALESCE(b.cat, 'n/a') apres LEFT JOIN sur cat_id = id
maintenance	VARCHAR	px_cat_g1v2	Indicateur maintenance, securise COALESCE a 'n/a'
cost	NUMERIC	prd_info.prd_cost	Cout unitaire nettoye (jamais negatif, jamais NULL)
product_line	VARCHAR	prd_info.prd_line	Gamme traduite : Road / Mountain / Touring / Other Sales
start_date	DATE	prd_info.prd_start_dt	Date d'introduction du produit sur le marche

5.4 Table de Faits : Gold.fact_sales

C'est le coeur transactionnel du modele en étoile. Elle centralise toutes les transactions de ventes et les raccorde aux dimensions via des jointures gauches sur les cle naturelles metiers.

Champ BI	Type SQL	Cle/Contrainte	Description
sale_id	VARCHAR	Cle transactionnelle	Numero unique du bon de commande
product_key	BIGINT	FK => Gold.dim_product	Jointure sur sls_prd_key = product_number
customer_key	BIGINT	FK => Gold.dim_customer	Jointure sur sls_cust_id = customer_key
order_date	DATE	Axe temporel	Date d'enregistrement de la commande

Champ BI	Type SQL	Cle/Contrainte	Description
ship_date	DATE	Axe temporel	Date d'expédition de la marchandise
due_date	DATE	Axe temporel	Date d'échéance de paiement ou de livraison
sale_amount	NUMERIC	Metric principale (CA)	Chiffre d'Affaires. Si manquant : Quantité x Prix
quantity	NUMERIC	Metric (Somme)	Nombre d'unités vendues, jamais négatif (ABS)
price	NUMERIC	Metric (Moy/Min/Max)	Prix unitaire facture, recalculé si incohérent

6. ORCHESTRATION GLOBALE

L'Orchestrateur.py est le point de controle maitre du projet. Place a la racine, il importe dynamiquement les trois modules enfants (Extract, Transform, Load) en ajoutant leurs dossiers respectifs au sys.path Python, puis execute le pipeline de bout en bout dans l'ordre Bronze > Silver > Gold.

6.1 Sequence d'execution

Etape	Module	Fonction appelee	Comportement sur echec
1/3	Bronze/Extract.py	pipeline_vers_bronze()	Arret du pipeline global, message d'erreur
2/3	Silver/Transform.py	run_silver_pipeline()	Rollback automatique psycopg2, arret du pipeline
3/3	Gold/Load.py	executer_generation_gold()	RuntimeError si script echoue (Fail-Fast)

6.2 Commande de lancement

```
# Configuration de la connexion
export DATABASE_URL="postgresql://user:password@host:port/dbname"
# Execution du pipeline complet
python Orchestrateur.py
```

7. SYSTEME D'AUDIT

Le projet inclut un schema d'audit complet (Audit.sql) qui cree une table de logs ultra-detaillée pour la gouvernance du pipeline. Cette table permet de tracer chaque execution du pipeline dans Power BI ou via des requetes SQL ad-hoc.

7.1 Structure de la table Audit.pipeline_logs

Colonne	Type	Description
log_id	SERIAL PK	Identifiant auto-incremente de chaque log
pipeline_name	VARCHAR(100)	Nom du projet global (ex: Sales_Intelligence_Pipeline)
script_name	VARCHAR(100)	Fichier exact execute (ex: stg_loc_a101.sql)
source_layer / target_layer	VARCHAR(50)	Couche de depart et couche d'arrivee (ex: Bronze > Silver)
load_type	VARCHAR(20)	Strategie : FULL REFRESH ou INCREMENTAL
execution_start/end_time	TIMESTAMP	Dates/heures precises de debut et fin du script
duration_seconds	NUMERIC(10,2)	Duree calculee en secondes pour analyse de performance
rows_inserted/updated/deleted	INT	Metriques de volumetrie pour graphiques de tendances
status	VARCHAR(20)	Etat : SUCCESS, FAILED ou RUNNING
error_message	TEXT	Message d'erreur SQL complet en cas de crash
executed_by	VARCHAR(100)	Identite du lanceur : python_job, airflow, manual_admin...

8. MODELE EN ETOILE — Schema Conceptuel

Le modele en etoile Gold suit rigoureusement la methodologie de Ralph Kimball. Les deux dimensions (dim_customer et dim_product) sont reliees a la table de faits centrale (fact_sales) par des relations 1-a-plusieurs basees sur des Surrogate Keys numeriques (BIGINT), ce qui maximise les performances de calcul dans le moteur VertiPaq de Power BI et minimise l'empreinte memoire.

dim_customer PK: customer_id (BIGINT) customer_key customer_number first_name last_name gender marital_status country birth_date create_date	1:N	fact_sales sale_id (VARCHAR) FK: product_key (BIGINT) FK: customer_key (BIGINT) order_date (DATE) ship_date (DATE) due_date (DATE) sale_amount (NUMERIC) quantity (NUMERIC) price (NUMERIC)	1:N	dim_product PK: product_key product_id product_number product_name category sub_category cost product_line start_date
---	-----	---	-----	---

9. GOUVERNANCE DES DONNEES

9.1 Regles de qualite appliquees

#	Regle	Implementation technique
1	Zero NULL en Gold	COALESCE() sur toutes les colonnes de filtrage des dimensions. Elimine les lignes vides dans les visuels Power BI.
2	Zero monetaire negatif	ABS() sur sls_sales, sls_quantity et sls_price dans Silver. Les negatifs de saisie humaine sont automatiquement corriges.
3	Coherence comptable	Si montant = 0 : recalcul via Quantite x Prix. Si prix = 0 : recalcul via Montant / Quantite. Garantit que Sales = Qty x Price sur 100% des lignes.
4	Dedoublonnage clients	ROW_NUMBER() OVER(PARTITION BY cst_id ORDER BY cst_create_date DESC). Un seul profil par client (le plus recent).
5	Normalisation géographique	Regroupement de tous les synonymes connus de pays en valeurs canoniques (US/USA/UNITED STATES => United States).
6	Securisation des dates	Validation de longueur (8 ou 10 caracteres selon le contexte) avant tout casting. Date pivot 1900-01-01 en cas d'anomalie.
7	Idempotence	DROP TABLE/VIEW IF EXISTS CASCADE sur chaque script. Le pipeline peut etre relance N fois avec toujours le meme resultat final.
8	SCD Type 2 Produits	LEAD() pour calculer prd_end_dt. Seules les versions actives (= '9999-12-31') sont exposees dans Silver, implementant un SCD1 en Gold.

9.2 Securite et deployment

- Credentials exclus du code source : DATABASE_URL lue uniquement via os.getenv() pour respecter les bonnes pratiques de securite (pas de mot de passe en clair dans les scripts).
- Prerequis systeme : pip install pandas sqlalchemy psycopg2-binary
- Compatibilite : PostgreSQL, Python 3.x, schemas 'Bronze', 'Silver', 'Gold' a creer manuellement si absents.
- Extensibilite : l'ajout d'une nouvelle source CSV ne necessite aucune modification de Extract.py (detection automatique de tous les *.csv dans datasets/).
- Compatibilite BI : les vues Gold sont conCues pour une connexion directe Power BI via le connecteur PostgreSQL natif, avec un modele semantique directement exploitable.

10. CONCLUSION

Ce projet represente une implementation de reference d'un Data Warehouse d'entreprise moderne. Il couvre l'integralite du cycle de vie de la donnee : de l'ingestion brute des fichiers sources jusqu'a l'exposition dans un modele en etoile performant, en passant par des etapes de nettoyage, de validation et de transformation rigoureusement documentees.

Les points de force architecturaux sont nombreux : idempotence totale a chaque etape, gestion transactionnelle avec rollback automatique, logique de fallback multi-sources pour maximiser la completude des dimensions, regles d'alignement comptable strictes sur les transactions financieres, et traçabilité par le schema d'audit. L'ensemble est pilote par un orchestrateur unique qui execute le pipeline de bout en bout en une seule commande.

Les pistes d'evolution naturelles incluent l'integration d'un outil d'orchestration comme Apache Airflow pour la planification, l'activation du schema d'audit pour monitorer les executions, l'ajout d'une dimension Temps (dim_date) pour les analyses temporelles avancees dans Power BI, et potentiellement la transition vers un mode incremental pour les tables de faits volumineuses.